Theory assignment – 3

Tanu B
261100690012

Exercise 1 – Identify Your Linux Filesystem

Explore the storage configuration of your Linux system.

Determine:

1. The storage devices available.
2. The partitions available on each device.
3. The filesystem type used by each partition.
4. The device/partition on which `/` is located.
5. The filesystem type used for `/`.
6. The total, used, and available storage space.

:
Storage devices available: The primary storage device is `/dev/sda` (a 40G disk). There is also a read-only optical drive `/dev/sr0` (51M) attached.

Partitions available: The `sda` disk is divided into three partitions: `sda1` (1M), `sda2` (513M), and `sda3` (39.5G).

Filesystem types:

sda2 uses **vfat** (FAT32).
sda3 uses **ext4**.
sr0 uses **iso9660**.

Partition for / (root): The root directory is located on **`/dev/sda3`**

Filesystem type for /: The root partition is formatted using the **ext4** filesystem

Total, used, and available storage space (for /):
Total Size: **39G**
Used: **21G** (56% utilization)
Available: **17G**

Device/Partition	Filesystem Type	Size	Mount Point
/dev/sda2	vfat	512M	/boot/efi
/dev/sda3	ext4	39G	/
/dev/sr0	iso9660	52M	/media/hdoop/VBox_GAs_7.0.16

```
loop23 squashfs 4.0                                                                    0   100% /snap/snapd-desktop-integration/391
sda
├─sda1
├─sda2 vfat     FAT32                           D623-05B6                            505.9M     1% /boot/efi
└─sda3 ext4     1.0                             d12ec526-388c-4b33-a8c7-6e8c337e6dbe   16.2G    53% /var/snap/firefox/common/host-hunspell
                                                                                                 /
sr0    iso9660  Joliet Extension VBox_GAs_7.0.16 2024-04-12-17-58-47-24                    0   100% /media/hdoop/VBox_GAs_7.0.16
hdoop@Ubuntu:~$ df -T
Filesystem     Type    1K-blocks     Used Available Use% Mounted on
tmpfs          tmpfs      793808     1868    791940   1% /run
/dev/sda3      ext4     40453376 21399716  16966544  56% /
tmpfs          tmpfs     3969036        0   3969036   0% /dev/shm
tmpfs          tmpfs        5120        4      5116   1% /run/lock
/dev/sda2      vfat       524252     6232    518020   2% /boot/efi
tmpfs          tmpfs      793804      132    793672   1% /run/user/1001
/dev/sr0       iso9660     52274    52274         0 100% /media/hdoop/VBox_GAs_7.0.16
hdoop@Ubuntu:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           776M  1.9M  774M   1% /run
/dev/sda3        39G   21G   17G  56% /
tmpfs           3.8G     0  3.8G   0% /dev/shm
tmpfs           5.0M  4.0K  5.0M   1% /run/lock
/dev/sda2       512M  6.1M  506M   2% /boot/efi
tmpfs           776M  132K  776M   1% /run/user/1001
/dev/sr0         52M   52M     0 100% /media/hdoop/VBox_GAs_7.0.16
hdoop@Ubuntu:~$ df -T /
Filesystem     Type 1K-blocks     Used Available Use% Mounted on
/dev/sda3      ext4  40453376 21399716  16966544  56% /
hdoop@Ubuntu:~$
```

Exercise 2 – Disk → Partition → Filesystem → Mount Point

Using the information obtained in Exercise 1, draw the storage organization of your own Linux system.
For example:

Disk
 |
 +-- Partition
      |
      +-- Filesystem
           |
           +-- Mount Point

Do not copy the example values. Use the actual values observed on your machine.
Explain in 2–3 sentences the relationship between these four components.

:
Disk (/dev/sda)
 |
 +-- Partition (/dev/sda2)
 | |
 | +-- Filesystem (vfat)
 | |
 | +-- Mount Point (/boot/efi)
 |
 +-- Partition (/dev/sda3)
      |
      +-- Filesystem (ext4)
           |
           +-- Mount Point (/)

A physical **disk** is divided into logical, isolated sections called **partitions** to manage different storage needs. Each partition is then formatted with a specific **filesystem** that dictates exactly how data is structured, indexed, and retrieved. Finally, the OS makes that structured data accessible by attaching it to a specific directory in the system's root hierarchy, known as a **mount point**.

Exercise 3 – Explore the Root Filesystem

Start from your current location.

1. Determine your current working directory. : **pwd**
2. Navigate to the root directory. : **cd /**
3. Display the contents of the root directory. : **ls**
4. Identify all directories directly available under `/`. : **ls -d /***
5. Determine whether hidden entries are present. : **ls -la /**

:

```
hdoop@Ubuntu:~$ pwd
/home/hdoop
hdoop@Ubuntu:~$ cd /
hdoop@Ubuntu:/$ pwd
/
hdoop@Ubuntu:/$ ls
bin  boot  cdrom  dev  etc  home  lib  lib32  lib64  libx32  lost+found  media  mnt  opt  proc  root  run  sbin  snap  srv  swapfile  sys  tmp  usr  var
hdoop@Ubuntu:/$ ls -d /*
/bin  /boot  /cdrom  /dev  /etc  /home  /lib  /lib32  /lib64  /libx32  /lost+found  /media  /mnt  /opt  /proc  /root  /run  /sbin  /snap  /srv  /swapfile  /sys  /tmp  /usr  /var
hdoop@Ubuntu:/$ ls -la /
total 2097240
drwxr-xr-x  20 root root       4096 Jun 30  2023 .
drwxr-xr-x  20 root root       4096 Jun 30  2023 ..
lrwxrwxrwx   1 root root          7 Jun 29  2023 bin -> usr/bin
drwxr-xr-x   4 root root       4096 Aug 28 00:32 boot
drwxrwxr-x   2 root root       4096 Jun 30  2023 cdrom
drwxr-xr-x  19 root root       4400 Sep  6 13:38 dev
drwxr-xr-x 138 root root      12288 Aug 29 20:14 etc
drwxr-xr-x   4 root root       4096 Jul  1  2023 home
lrwxrwxrwx   1 root root          7 Jun 29  2023 lib -> usr/lib
lrwxrwxrwx   1 root root          9 Jun 29  2023 lib32 -> usr/lib32
lrwxrwxrwx   1 root root          9 Jun 29  2023 lib64 -> usr/lib64
lrwxrwxrwx   1 root root         10 Jun 29  2023 libx32 -> usr/libx32
drwx------   2 root root      16384 Jun 29  2023 lost+found
drwxr-xr-x   4 root root       4096 Jul  1  2023 media
drwxr-xr-x   2 root root       4096 Feb 23  2023 mnt
drwxr-xr-x   3 root root       4096 Jun 30  2023 opt
dr-xr-xr-x 280 root root          0 Sep  6 13:23 proc
drwx------   6 root root       4096 May  1  2024 root
drwxr-xr-x  37 root root       1020 Sep  6 13:38 run
lrwxrwxrwx   1 root root          8 Jun 29  2023 sbin -> usr/sbin
drwxr-xr-x  18 root root       4096 Aug 13 21:05 snap
drwxr-xr-x   2 root root       4096 Feb 23  2023 srv
-rw-------   1 root root 2147483648 Jun 29  2023 swapfile
dr-xr-xr-x  13 root root          0 Sep  6 13:23 sys
drwxrwxrwt  19 root root       4096 Sep  6 14:30 tmp
drwxr-xr-x  14 root root       4096 Feb 23  2023 usr
drwxr-xr-x  14 root root       4096 Feb 23  2023 var
hdoop@Ubuntu:/$
```

Exercise 4 – Linux Filesystem Hierarchy Investigation

Investigate the following locations:

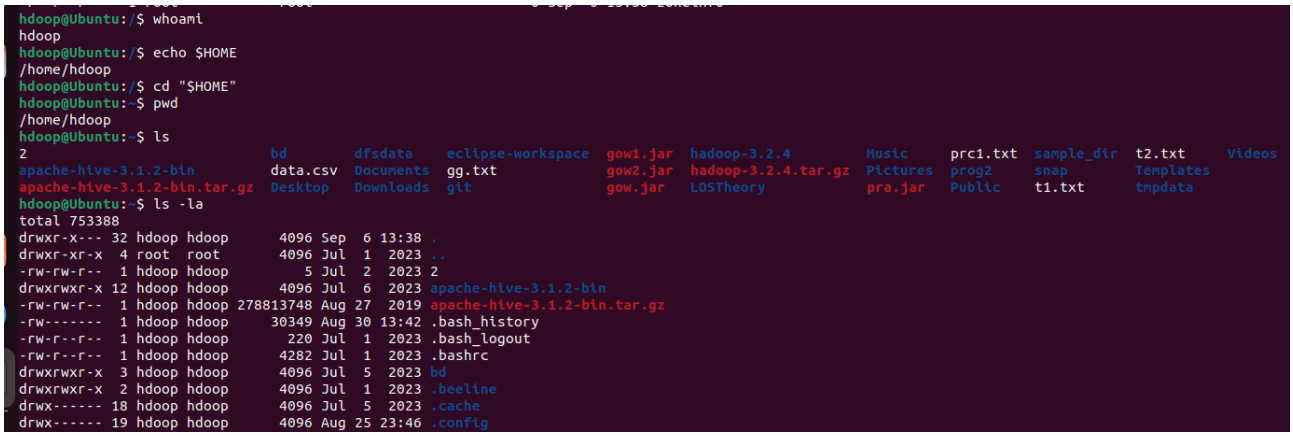
/home
/root
/etc
/var
/var/log
/tmp
/usr
/boot
/dev
/proc

For each location:
1. Examine its contents.
2. Identify at least three entries, where available.
3. Determine the likely purpose of the directory.

Directory	Example Contents	Purpose
/home	hdoop, vboxuser	Stores the personal directories and user-specific configuration files for standard, non-root users.
/root	.bashrc, .bash_history, vboxpostinstall.sh	Serves as the specific home directory designated for the root (superuser) account.
/etc	passwd, fstab, hostname	Stores core system-wide configuration files, startup scripts, and network settings.
/var	log, lib, cache	Contains variable data files that are expected to grow and change continuously during normal system operation (such as databases and spools).
/var/log	syslog, auth.log, kern.log	Acts as the centralized storage location for system, kernel, authentication, and application log files.
/tmp	snap-private-tmp, systemd-private-..., .X11-unix	Provides a world-writable directory for temporary files created by the system and users, which is typically cleared upon reboot.
/usr	bin, lib, share	Contains installed user utilities, applications, shared libraries, and static read-only data for user-level programs.
/boot	grub, vmlinuz-6.8.0-138-generic, initrd.img...	Stores the essential files required for the system boot process, including the Linux kernel and bootloader configurations.
/dev	sda, null, tty	Contains special device nodes that act as interfaces between user-space software and the system's physical or virtual hardware.
		Operates as a dynamic virtual

Exercise 5 – Find Your Home Directory

Without assuming its location:
1. Determine your username. : **whoami**
2. Determine your home directory. : **echo $HOME**
3. Navigate to your home directory. : **cd "$HOME"**
4. Display its absolute path. : **pwd**
5. List its normal contents. : **ls**
6. List all contents, including hidden files. : **ls -la**



Exercise 6 – Hidden Files Investigation

Examine your home directory.
Compare:

ls
ls -a

Identify at least 2 hidden files/directories.
Answer:

1. How does Linux identify a hidden file?
: Linux identifies a file or directory as hidden if its name **begins** with a period or **"dot"** (e.g., .bashrc).

2. Why does a normal `ls` not display these files?
: The ls command is programmed by default to filter out any files starting with a dot. This is done to declutter the terminal output, as hidden files are usually configuration files, history logs, or cache directories that standard users don't need to see during routine file browsing.

3. What do `.` and `..` mean?

- : . (single dot) represents the **current working directory** you are in.
- .. (double dot) represents the **parent directory** (one level up in the hierarchy).

4. Can a hidden file still be accessed if its name is known?
: Yes. Hidden files are not inherently restricted or locked; they are only visually hidden from default listings. If you know the name (or use ls -a to find it), you can read, edit, or execute them just like any normal file (e.g., cat .bash_history).

5. From a cybersecurity perspective, why should hidden files not automatically be considered safe or malicious?
: They should not be automatically considered **safe** because attackers frequently use the dot-prefix to hide malware, backdoors, or staging folders from casual administrators who only run basic ls commands. Conversely, they should not automatically be considered **malicious** because almost all legitimate Linux applications and user environments rely extensively on hidden files to store essential configurations and local settings (like .ssh/authorized_keys or .profile). Their safety must be determined by their contents, permissions, and location, not just their hidden status.

: Identified Hidden Files/Directories:

- .bash_history (Hidden file)
- .cache (Hidden directory)
- .config (Hidden directory)
- .ssh (Hidden directory)

Exercise 7 – Build a Small Filesystem Hierarchy

Explore: mkdir

Hint: you can create this hierarchy in a single command

cyber_lab/

|

+-- evidence/

| +-- network/

| +-- system/

|

+-- logs/

|

+-- scripts/

|

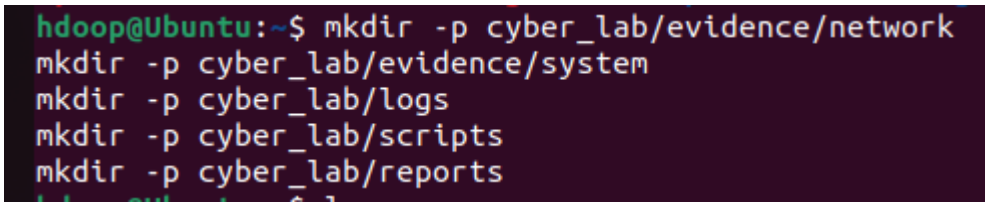
+-- reports/

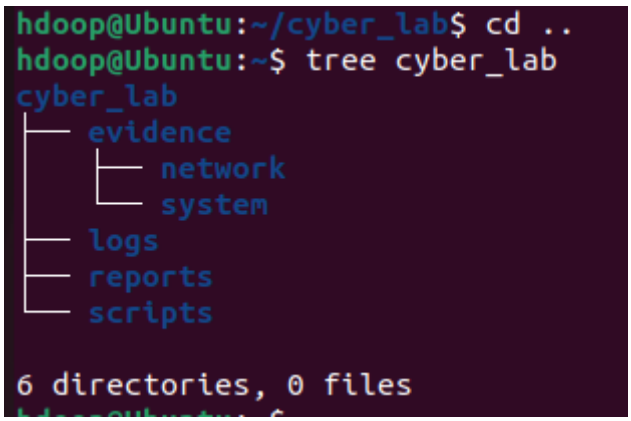
```
```

Create the following files:

cyber_lab/evidence/network/traffic.txt
cyber_lab/evidence/system/processes.txt
cyber_lab/logs/security.log
cyber_lab/scripts/check.sh
cyber_lab/reports/investigation.txt

Use appropriate Linux commands to create the structure.
Verify that the required hierarchy has been created successfully.
Hint: tree
Explore options





```
hdoop@Ubuntu:~$ touch cyber_lab/evidence/network/traffic.txt
touch cyber_lab/evidence/system/processes.txt
touch cyber_lab/logs/security.log
touch cyber_lab/scripts/check.sh
touch cyber_lab/reports/investigation.txt
hdoop@Ubuntu:~$ tree cyber_lab
cyber_lab
├── evidence
│   ├── network
│   │   └── traffic.txt
│   └── system
│       └── processes.txt
├── logs
│   └── security.log
├── reports
│   └── investigation.txt
└── scripts
    └── check.sh

6 directories, 5 files
```

Exercise 8 – Absolute and Relative Paths

Assume your current directory is:

~/cyber_lab

Perform the following:

1. Navigate to `evidence/network` using a relative path. : **cd evidence/network**
2. Navigate to `/etc` using an absolute path. : **cd /etc**
3. Return to your home directory using the shortest possible command. : **cd**
4. Navigate to `cyber_lab/reports`. : **cd ~/cyber_lab/reports**
5. Move one directory upward. : **cd ..**
6. Display the current directory after every operation.

*** What is an absolute path?**
: An absolute path is a f**ile or directory's full and exact location starting from** the very top of the filesystem hierarchy—**the root directory (/)**. It always **begins with a /** and remains valid no matter what your current working directory is (e.g., /home/hdoop/cyber_lab/reports).

*** What is a relative path?**
: A relative path is a **file or directory's location defined in relation to your *current working directory***. It does not start with a /. For example, if you are inside cyber_lab, typing cd reports uses a relative path to move into the subdirectory.

*** What is the difference between `/` and `~`?**

- / represents the **root directory**, which is the absolute top-level directory of the entire Linux filesystem. Every single file and folder on the system sits somewhere under /.
- ~ (tilde) is a shell shortcut representing the **current user's home directory**. For your account, ~ is automatically translated by the system to /home/hdoop.

Exercise 9 – Identify File Types

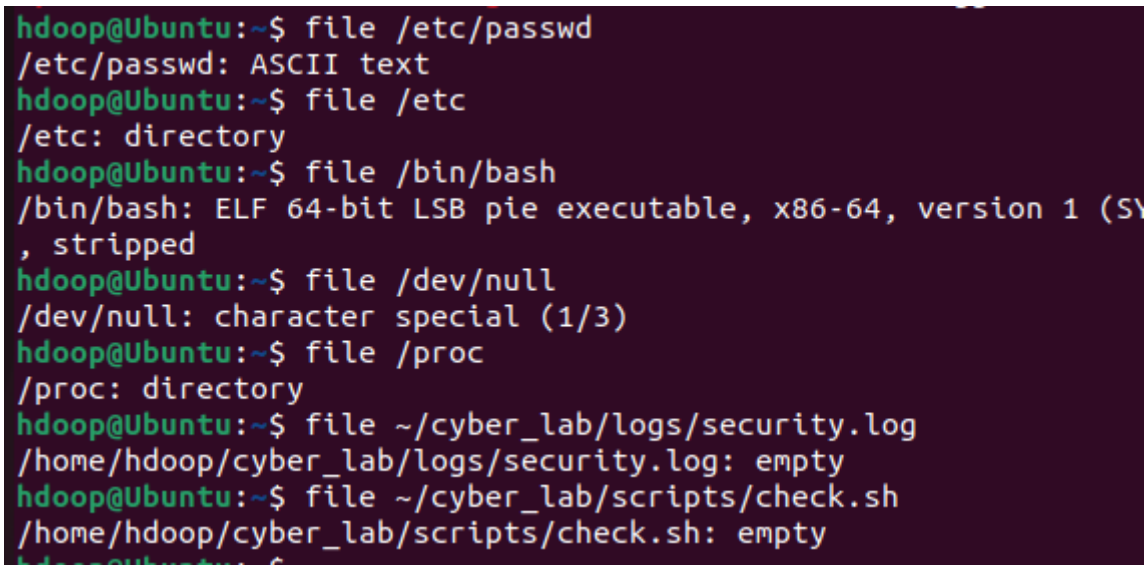
Investigate the following:
/etc/passwd
/etc
/bin/bash
/dev/null
/proc

Also investigate the files created in Exercise 7.

file <filename>
ls -l <filename>

Classify each object.

Object	**Type (from file command)**	**Type (from ls -l first character)**
/etc/passwd	ASCII text	Regular file (-)
/etc	Directory	Directory (d)
/bin/bash	ELF 64-bit LSB executable	Regular file (-)
/dev/null	Character special	Character device (c)
/proc	Directory	Directory (d)
security.log	Empty file	Regular file (-)
check.sh	Empty file *(or Bourne-Again shell script if you added code)*	Regular file (-)



Exercise 10 – Examine File Metadata

Use:
~/cyber_lab/reports/investigation.txt

Add a small amount of text to the file.

Examine it using:
ls -l
stat investigation.txt

Identify:

Filename: investigation.txt

- **File size:** 32 (bytes)
- **File type:** regular file
- **Owner:** hdoop (Uid: 1001)
- **Group:** hdoop (Gid: 1001)
- **Inode number:** 1721682
- **Access time:** 2026-09-06 16:28:50.299921140 +0530
- **Modification time:** 2026-09-06 16:28:35.115921396 +0530
- **Change time:** 2026-09-06 16:28:35.115921396 +0530

1. What is metadata?
: Metadata is "data about data." In a Linux filesystem, it refers to the system attributes that describe a file—such as its size, permissions, owner, physical location on the disk (inode), and timestamps—rather than the actual text or data stored inside the file.

2. Is metadata the same as file content?

: No. File content is the actual data you put into the file (in this case, the text "Filesystem investigation report"). Metadata is the administrative information the operating system uses to manage and track that file.

3. Which timestamp changes when file contents are modified?
: The Modification time (Modify/mtime) changes whenever the actual contents of the file are altered. (Note: The Change time (Change/ctime) will also update concurrently, because altering the contents usually changes the file's size, which is a change to its metadata).

```
hdoop@Ubuntu:~$ cd ~/cyber_lab/reports
hdoop@Ubuntu:~/cyber_lab/reports$ echo "Filesystem investigation report" > investigation.txt
hdoop@Ubuntu:~/cyber_lab/reports$ cat investigation.txt
Filesystem investigation report
hdoop@Ubuntu:~/cyber_lab/reports$ ls -l investigation.txt
-rw-rw-r-- 1 hdoop hdoop 32 Sep  6 16:28 investigation.txt
hdoop@Ubuntu:~/cyber_lab/reports$ stat investigation.txt
  File: investigation.txt
  Size: 32              Blocks: 8          IO Block: 4096   regular file
Device: 803h/2051d      Inode: 1721682     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1001/   hdoop)   Gid: ( 1001/   hdoop)
Access: 2026-09-06 16:28:50.299921140 +0530
Modify: 2026-09-06 16:28:35.115921396 +0530
Change: 2026-09-06 16:28:35.115921396 +0530
 Birth: 2026-09-06 16:00:54.476949473 +0530
```

Exercise 11 – Explore `/etc`

Investigate:
/etc

Locate:
passwd
group
hostname
hosts

Note:

```
hdoop@Ubuntu:~/cyber_lab/reports$ ls -l /etc/passwd
-rw-r--r-- 1 root root 3031 Jul  1  2023 /etc/passwd
hdoop@Ubuntu:~/cyber_lab/reports$ ls -l /etc/group
-rw-r--r-- 1 root root 1104 Jul  1  2023 /etc/group
hdoop@Ubuntu:~/cyber_lab/reports$ ls -l /etc/hostname
-rw-r--r-- 1 root root 7 Jun 30  2023 /etc/hostname
hdoop@Ubuntu:~/cyber_lab/reports$ ls -l /etc/hostname
-rw-r--r-- 1 root root 7 Jun 30  2023 /etc/hostname
hdoop@Ubuntu:~/cyber_lab/reports$ ls -l /etc/hosts
-rw-r--r-- 1 root root 251 Jun 30  2023 /etc/hosts
```

Without modifying any system files, examine their contents where permitted.

Answer:

1. What kind of information is primarily stored under `/etc`?
: The /etc directory primarily stores core, system-wide configuration files, startup scripts, and administrative settings. It does not contain executable programs, but rather the text files that dictate how the system and its applications behave.

2. What information does `/etc/passwd` contain?
: It contains a list of all user accounts on the Linux system. Each line represents a single user (e.g., hdoop:x:1001:1001:,,,:/home/hdoop:/bin/bash) and includes their username, an 'x' indicating their password is encrypted elsewhere, their User ID (UID), Group ID (GID), user information/comments, home directory path, and default login shell.

3. What information does `/etc/hostname` contain?
: It contains the simple string representing the system's network name. Based on your output, your machine's hostname is exactly Ubuntu.

4. Why is `/etc` security-sensitive?
: Because it contains the blueprints for how the system operates. If a malicious actor or unauthorized user can modify files in /etc, they could easily create fake root accounts in /etc/passwd, alter network routing in /etc/hosts, disable security controls, or manipulate startup scripts to establish persistent malware. Therefore, almost all files in /etc require root privileges to modify.

Exercise 12 – Explore Linux Log Storage

Investigate:
/var/log
: **ls -la /var/log**

Identify:
* Available log files : **find /var/log -maxdepth 1 -type f -print**
* Available log directories : **find /var/log -maxdepth 1 -type d -print**
* Recently modified entries : **ls -lt /var/log**
* Entries you can read : **find /var/log -type f -readable -print**
* Entries you cannot read as a normal user : **find /var/log -type f ! -readable -print**

Answer:

1. Why does Linux maintain logs?
: Linux maintains logs to create a continuous, historical record of system events, application behavior, and user activity. This allows administrators to monitor system health, troubleshoot hardware or software failures, audit security policies, and ensure services are running correctly.

2. Why are logs useful during incident investigation?
: Logs provide the critical audit trail necessary to reconstruct an incident. They allow investigators to establish a timeline of events, identify compromised accounts, trace an attacker's steps (such as finding failed SSH login attempts or malicious web requests), and determine the scope and impact of a breach.

3. Why might access to some logs be restricted?
: Logs frequently contain sensitive information, such as IP addresses, user command histories, authentication tokens, or internal system configurations. Restricting access (typically limiting it to the root user or the adm group) prevents standard users from spying on others and makes it harder

for unauthorized users or attackers to read sensitive data or easily cover their tracks by altering the logs.

Exercise 13 – Investigate `/tmp`

Explore `/tmp` without modifying files belonging to other users.
ls -la /tmp

Determine:
1. Whether files/directories currently exist.
: find /tmp -maxdepth 1 -type d -print , find /tmp -maxdepth 1 -type f -print

2. Whether hidden files exist.
: ls -la /tmp

3. Who owns some of the entries.
: ls -la /tmp

4. Which entries were modified recently.
: **ls -lt /tmp**

Why can temporary directories be relevant during a cybersecurity investigation?
: The /tmp directory is world-writable, meaning any user or process on the system can create, edit, and execute files there without needing root or elevated privileges. This makes it a prime, low-friction staging ground for attackers. During an intrusion, threat actors frequently use /tmp to drop exploit scripts, download malware payloads, run cryptominers, or temporarily aggregate and compress sensitive data before exfiltrating it off the network. Because of this, investigators often check /tmp to uncover malicious artifacts, leftover scripts, or tools the attacker failed to clean up.

Exercise 14 – Explore the `/proc` Filesystem

Investigate:
/proc

Examine:
cat /proc/cpuinfo
cat /proc/meminfo

Then identify your current shell PID:
echo $$: 4147

Check whether a corresponding directory exists under `/proc`.

Answer:

```
HugePages_Surp:        0
Hugepagesize:       2048 kB
Hugetlb:               0 kB
DirectMap4k:      120768 kB
DirectMap2M:     8071168 kB
hdoop@Ubuntu:~/cyber_lab/reports$ echo $$
4147
hdoop@Ubuntu:~/cyber_lab/reports$ ls -ld /proc/4147
dr-xr-xr-x 9 hdoop hdoop 0 Sep  6 13:44 /proc/4147
hdoop@Ubuntu:~/cyber_lab/reports$
```

1. What information does `/proc/cpuinfo` provide?

: /proc/cpuinfo provides detailed hardware specifications about the system's processor(s). According to your output, it shows you have a GenuineIntel processor (11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz) with 3 active cores, a clock speed of ~2803 MHz, and a 12288 KB cache size. It also lists supported CPU flags and known hardware bugs (like spectre_v1).

2. What information does `/proc/meminfo` provide?

: /proc/meminfo provides a real-time, comprehensive breakdown of system memory utilization. Your output shows the total physical RAM (MemTotal: ~7.9 GB), free RAM (MemFree: ~3.9 GB), available RAM (MemAvailable: ~6.1 GB), and swap space usage (SwapTotal: ~2 GB, all of which is free). It also details how memory is allocated for caching, buffers, and system structures.

3. Why do directories with numbers appear inside `/proc`?

: The numbered directories (like 1, 17, and your current shell 4147) represent the Process IDs (PIDs) of every actively running process on the system. Each directory contains live status, memory mapping, and execution details for that specific process.

4. Is `/proc` an ordinary disk directory?

: No, /proc is not an ordinary disk directory. It is a pseudo-filesystem (procfs) that resides completely in RAM. It acts as an interface directly to the kernel, providing a window into the internal state of the operating system.

5. Why could `/proc` be useful during security investigation?

: Since /proc displays live system state, it is invaluable for incident response and forensics. Investigators can examine running processes in real-time to uncover hidden malware that has deleted itself from the disk but is still running in memory. It allows for the extraction of process command-line arguments, environment variables, active network connections, and open file handles associated with suspicious PIDs.

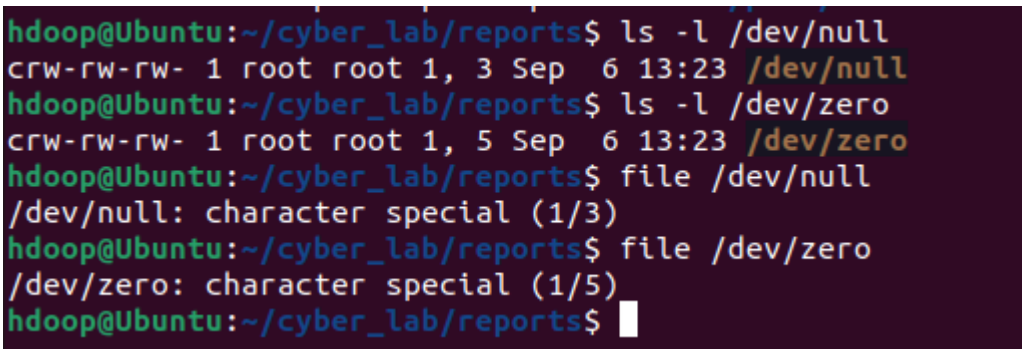
Exercise 15 – Explore `/dev`

/dev

Examine:

ls -l /dev/null
ls -l /dev/zero

1. What does `/dev` represent?



: /dev contains special device files (or device nodes). It acts as the interface between user-space applications and the system's hardware or virtual kernel drivers.

2. Are the entries ordinary user files?
: No, they are not ordinary text or binary files that store data on a hard drive. As shown by the c at the beginning of the ls -l output, they are special device nodes that act as communication channels to the kernel.

3. Determine the file type of `/dev/null`.
: According to the file command output and the leading c in the ls output, /dev/null is a **character special file** (also known as a character device).

4. Why does Linux expose devices through the filesystem?
: This follows the core Unix design philosophy that "everything is a file." By exposing hardware and virtual devices as files in /dev, Linux allows standard programs and commands (like echo, cat, or dd) to interact with hardware natively using standard read/write operations, without requiring specialized, device-specific APIs or software.

Exercise 16 – Filesystem Space Investigation

Determine:
1. Total filesystem capacity.
2. Used space.
3. Available space.
4. Percentage of filesystem utilization.
5. Space occupied by your home directory.
6. Space occupied by the `cyber_lab` directory.

Use appropriate combinations of:
df
df -h
du
du -h
du -sh

Answer:
Explain the difference between `df` and `du`.

- df (disk free) reports the total amount of free and used disk space at the macro filesystem level (like /dev/sda3). It gets this information instantly by reading the filesystem's metadata (superblock indices).

- du (disk usage) calculates the actual physical space occupied by specific directories and files. It does this recursively, meaning it has to open and read the size of every single file inside the target folder (like ~/cyber_lab) to calculate a total sum.

Exercise 17 – Cybersecurity Investigation Scenario

Assume that you are investigating a Linux machine after suspicious activity has been reported. You are initially asked to examine only the filesystem.

Identify which of the following locations you would investigate and explain **why**:

/home
ls -la /home
find /home -type f -mtime -7 -ls

Why: This directory holds individual user profiles, documents, and personal configurations. Attackers often compromise standard user accounts first before attempting privilege escalation.

Expected Information: By running find /home -type f -mtime -7 -ls, you are actively searching for files modified within the last 7 days. You would expect to find unauthorized SSH keys (.ssh/authorized_keys), modified shell profiles (.bashrc or .profile) used for persistence, or suspicious commands executed by the user in .bash_history.

/etc
ls -la /etc
cat /etc/passwd
cat /etc/hostname
cat /etc/hosts

Why: This is the core configuration hub of the Linux system. System-wide settings, network routing, and authentication parameters are stored here.

Expected Information: By examining passwd, you look for newly created, unauthorized user accounts or accounts unexpectedly granted root (0:0) privileges. Checking hosts can reveal malicious DNS redirects. You would also look for tampered service configurations (like altered sshd_config or malicious cron tasks).

/var/log
ls -lt /var/log
find /var/log -type f -readable -print

Why: This is the primary repository for system event data. It provides the historical timeline necessary to reconstruct an attack.

Expected Information: Sorting logs by time (ls -lt) allows you to correlate system events with the reported time of the suspicious activity. You would look for repeated failed logins (brute-force attacks), unauthorized successful sudo executions, application crashes indicating exploitation, or gaps in the logs that suggest an attacker attempted to clear their tracks.

/tmp
ls -lt /var/log
find /var/log -type f -readable -print

Why: The temporary directory is world-writable. Any process or user, regardless of privilege level, can write files here, making it the most common staging ground for initial payloads.

Expected Information: You would look for leftover exploit scripts, downloaded malware binaries, dropped rootkits, or compressed .tar.gz archives of sensitive data that an attacker has staged for exfiltration.

/proc
ps aux
ls /proc
cat /proc/cpuinfo
cat /proc/meminfo

Why: This is a dynamic, in-memory virtual filesystem that interfaces directly with the kernel. It holds real-time data about active system state and running processes.

Expected Information: Using ps aux alongside /proc directory inspection helps identify malicious processes currently executing in memory. You might find hidden processes, fileless malware holding open network ports, or anomalies in resource consumption (using cpuinfo or meminfo to spot cryptominers).

/dev
ls -la /dev
ls -l /dev/null
ls -l /dev/zero

Why: This directory contains device nodes representing hardware.

Expected Information: While less common for initial staging, advanced attackers might create rogue device files to bypass normal filesystem monitoring, install keyloggers attached to input

devices (/dev/input), or use /dev/null or /dev/zero to quietly discard output from malicious scripts running in the background.

/root
sudo ls -la /root

Why: This is the administrative superuser's home directory. Any unauthorized activity here confirms a total system compromise (privilege escalation to root).

Expected Information: Running sudo ls -la /root allows you to hunt for the same artifacts as in /home, but with much higher severity. You would look for attacker toolkits left behind, malicious root .bash_history commands, or backdoor SSH keys planted to guarantee persistent administrative access.